

1. مفهوم فرق تسد (Divide and Conquer)

تعتمد هذه الاستراتيجية على ثلاث خطوات أساسية:

- **التقسيم (Divide):** تقسيم المشكلة الكبيرة إلى مشكلات فرعية أصغر من نفس النوع.
- **التغلب (Conquer):** حل المشكلات الفرعية بشكل مستقل (عادةً عن طريق الاستدعاء الذاتي/Recursion).
- **الدمج (Combine):** دمج حلول المشكلات الفرعية للوصول إلى الحل النهائي للمشكلة الأصلية.
- **ملاحظة:** تتطلب هذه الخوارزميات دائماً وجود "حالة أساس" (Base Case) لإيقاف التكرار.

2. البحث الثنائي: الخطوات الرياضية (Binary Search - Slide 27)

يعتمد البحث الثنائي على مبدأ تقليص مساحة البحث إلى النصف في كل خطوة، ويشترط أن تكون المصفوفة مرتبة:

- **المقارنة:** يتم مقارنة العنصر المطلوب (x) بالعنصر الموجود في منتصف المصفوفة (mid).
- **القرار الرياضي:**
 - إذا كان العنصر في المنتصف يساوي x، يتم إنهاء البحث بنجاح.
 - إذا كان x أصغر من عنصر المنتصف، يتم إهمال النصف الأيمن والبحث في النصف الأيسر فقط.
 - إذا كان x أكبر من عنصر المنتصف، يتم إهمال النصف الأيسر والبحث في النصف الأيمن فقط.

Binary search (4/5)

```
FUNCTION recursiveBinarySearch(A, x, low, high)
  // Base Case 1: Search space is invalid
  IF low > high THEN
    RETURN -1
  END IF

  mid = low + (high - low) // 2

  // Base Case 2: Target found
  IF A[mid] == x THEN
    RETURN mid
  // Recursive Case 1: Search left subarray
  ELSE IF x < A[mid] THEN
    RETURN recursiveBinarySearch(A, x, low, mid - 1)
  // Recursive Case 2: Search right subarray
  ELSE
    RETURN recursiveBinarySearch(A, x, mid + 1, high)
  END IF
END FUNCTION
```

Page - 29

Recurrence Relation:

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

- **Time Complexity: $O(\log n)$ by master method**
 - Best Case: $O(1)$. The target is the first element.
 - Worst Case: $O(\log n)$. The target is the last element or not present. Every element must be inspected.
 - Average Case: $\theta(\log n)$. On average, we expect to check $n/2$ elements.



3. التعقيد الزمني: البحث الخطي مقابل البحث الثنائي

- **البحث الخطي (Linear Search):**

Linear search (2/2)

```
FUNCTION LinearSearch(A, x)
  // A: the sorted array of size n
  // x: the target value to find

  n = length(A)
  FOR i FROM 0 TO n-1 DO
    IF A[i] == x THEN
      RETURN i    // Target found at index i
    END IF
  END FOR
  RETURN -1      // Target not found
END FUNCTION
```

✓ Time Complexity: $O(n)$

- Best Case: $\Omega(1)$. The target is the first element.
- Worst Case: $O(n)$. The target is the last element or not present. Every element must be inspected.
- Average Case: $\Theta(n)$. On average, we expect to check $\sim n/2$ elements.



4. لماذا البحث الثنائي أفضل من البحث الخطي؟

- البحث الثنائي أسرع بكثير في التعامل مع البيانات الضخمة؛ فمثلاً في مصفوفة تحتوي على مليار عنصر، قد يحتاج البحث الخطي إلى مليار مقارنة، بينما يحتاج البحث الثنائي إلى 30 مقارنة فقط كحد أقصى.
- العيب الوحيد: البحث الثنائي يتطلب أن تكون البيانات مرتبة مسبقاً، بينما البحث الخطي يعمل على أي بيانات.

5. التعقيد الزمني لجميع الخوارزميات والأساليب في المحاضرة 5

- إيجاد القيمة العظمى والصغرى (Min & Max):
 - أسلوب الحل المباشر (Brute Force): التعقيد الزمني هو $O(n)$.

Brute Force Approach

- The brute force approach is straightforward: iterate through the entire array once, comparing each element to the current maximum and current minimum, updating them as needed.
- **Time Complexity: $O(n)$.**

```
FUNCTION findMaxMinBruteForce(A)
    // A: input array of size n
    n = length(A)
    IF n == 0 THEN
        RETURN NULL, NULL // Handle empty array
    END IF

    max = A[0]
    min = A[0]

    FOR i FROM 1 TO n-1 DO
        IF A[i] > max THEN
            max = A[i] // Update max if current element is larger
        END IF
        IF A[i] < min THEN
            min = A[i] // Update min if current element is smaller
        END IF
    END FOR

    RETURN max, min
END FUNCTION
```

Page - 33

○ أسلوب "فرق تسد" (Conquer & Divide): التعقيد الزمني هو $O(n)$ أيضاً، وعلاقة التكرار هي: $T(n) = 2T(n/2) + 1$.

Divide and Conquer Approach

- Recurrence Relation: $T(n) = 2T(\frac{n}{2}) + 1$
- According to master method, time complexity is $O(n)$.

```
// Recursive helper function
FUNCTION findMaxMinRecursive(A, low, high)
    // Base case: single element
    IF low == high THEN
        RETURN A[low], A[low] // max = min = single element
    END IF

    // Base case: two elements
    IF high == low + 1 THEN
        IF A[low] > A[high] THEN
            RETURN A[low], A[high]
        ELSE
            RETURN A[high], A[low]
        END IF
    END IF

    // Divide: split the array into two halves
    mid = low + (high - low) // 2

    // Conquer: recursively find max/min of left and right halves
    leftMax, leftMin = findMaxMinRecursive(A, low, mid)
    rightMax, rightMin = findMaxMinRecursive(A, mid + 1, high)

    // Combine: compare results from left and right halves
    overallMax = MAX(leftMax, rightMax)
    overallMin = MIN(leftMin, rightMin)

    RETURN overallMax, overallMin
END FUNCTION
```

Page - 35

• حساب القوة (a^n - Powering a number):

○ أسلوب الحل المباشر (Brute Force): يتم ضرب العدد في نفسه n من المرات، والتعقيد هو $O(n)$.

Brute Force Approach

- The brute force approach multiplies a by itself n times using iterative multiplication.
- **Time Complexity: $O(n)$.**

```
FUNCTION powerNaive(a, n)
  // a: base number
  // n: non-negative exponent

  IF n == 0 THEN
    RETURN 1
  END IF

  result = 1
  FOR i FROM 1 TO n DO
    result = result * a
  END FOR

  RETURN result
END FUNCTION
```

Page - 40



- أسلوب "فرق تسد" (Conquer & Divide): يتم تقسيم القوة إلى النصف في كل مرة، والتعقيد هو $O(\log n)$ ، وعلاقة التكرار هي: $T(n) = T(n/2) + 1$.

Divide and Conquer Approach

- The divide and conquer approach uses the mathematical insight that:

$$\text{If } n \text{ is even: } a^n = a^{\frac{n}{2}} \cdot a^{\frac{n}{2}}$$

$$\text{If } n \text{ is odd: } a^n = a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a$$

- This reduces the problem size by half at each step.

Page - 41



Divide and Conquer Approach

- Recurrence Relation: $T(n) = T\left(\frac{n}{2}\right) + 1$
- According to master method, time complexity is $O(\log n)$.

```
FUNCTION powerDivideConquer(a, n)
    // a: base number
    // n: non-negative exponent

    // Base case
    IF n == 0 THEN
        RETURN 1
    END IF

    // Recursive case
    half = powerDivideConquer(a, n // 2) // Integer division

    IF n % 2 == 0 THEN
        // n is even
        RETURN half * half
    ELSE
        // n is odd
        RETURN half * half * a
    END IF
END FUNCTION
```

Page - 42

ملاحظة ختامية:

في عملية حساب القوة (Power)، يعتبر أسلوب "فرق تسد" أفضل بمراحل من الحل المباشر، بينما في إيجاد العظمى والصغرى (Min & Max)، يتساويان في التعقيد الزمني ($O(n)$)، لكن الحل المباشر قد يكون أفضل قليلاً في التطبيق العملي لعدم وجود استدعاءات ذاتية (Recursion) تستهلك الذاكرة.

Lecture 6

1. ##### الخطوات الرياضية للخوارزميات (Mathematical Steps)

**** خوارزمية ترتيب الفقاعة (Bubble Sort): ****

*تقوم الخوارزمية بالمرور المتكرر عبر القائمة.

*تقارن كل عنصر بالعنصر الذي يليه (المتجاورة) وتقوم بتبديلهما إذا كانا في ترتيب خاطئ.

*تستمر التمريرات حتى تكتمل دورة كاملة بدون أي تبديل، مما يعني أن القائمة مرتبة.

Bubble Sort (3/3)

```
FUNCTION bubbleSort(A)
  // A: the array to be sorted
  n = length(A)
  FOR i FROM 0 TO n-2 DO
    swapped = false
    FOR j FROM 0 TO n-i-2 DO //Last i elements are already in place
      IF A[j] > A[j+1] THEN
        SWAP(A[j], A[j+1])
        swapped = true
      END IF
    END FOR
    // If no two elements were swapped, the array is sorted.
    IF NOT swapped THEN
      BREAK
    END IF
  END FOR
END FUNCTION
```

● Time Complexity: $O(n^2)$

- Worst Case: $O(n^2)$ - The array is in reverse order. We need n passes, each doing $\sim n$ comparisons.
- Best Case: $O(n)$ - The array is already sorted. The optimized version (with swapped flag) will only do one pass.
- Average Case: $O(n^2)$



**** خوارزمية الترتيب بالدمج (Merge Sort): ****

**** التقسيم (Divide): **** تقسيم المصفوفة إلى نصفين بشكل متكرر حتى نصل إلى مصفوفات تحتوي على عنصر واحد.

**** التغلب (Conquer): **** ترتيب هذه المصفوفات الصغيرة.

**** الدمج (Merge): **** دمج المصفوفات المرتبة مع بعضها البعض لتكوين مصفوفة واحدة كبيرة مرتبة.

Merge Sort (3/5)

- Recurrence Relation: $T(n) = 2T(\frac{n}{2}) + n$
 - ✓ $2T(\frac{n}{2})$: Time to recursively sort two subarrays of size $n/2$.
 - ✓ n Time to merge the two sorted subarrays.
- Time Complexity: $(n \log n)$ in all cases (worst, average, best), calculated by master method.

```
def merge_sort(A, low, high):
    # Base case: a subarray of length 1 or 0 is already sorted
    if low >= high:
        return

    # Recursive case: divide the array into two halves
    mid = (low + high) // 2

    # Recursively sort the left half
    merge_sort(A, low, mid)

    # Recursively sort the right half
    merge_sort(A, mid + 1, high)

    # Merge the two sorted halves
    merge(A, low, mid, high)

def merge(A, low, mid, high):
    # Create temporary arrays for left and right halves
    left = A[low:mid+1]
    right = A[mid+1:high+1]

    i = j = 0
    k = low

    # Merge the two arrays by comparing elements
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            A[k] = left[i]
            i += 1
        else:
            A[k] = right[j]
            j += 1
        k += 1

    # Copy any remaining elements from left array
    while i < len(left):
        A[k] = left[i]
        i += 1
        k += 1

    # Copy any remaining elements from right array
    while j < len(right):
        A[k] = right[j]
        j += 1
        k += 1
```

Page - 10

**** خوارزمية الترتيب السريع (Quick Sort) ****

**** اختيار المحور (Pivot): **** اختيار عنصر من المصفوفة ليكون مرجعاً.

**** التجزئة (Partitioning): **** وضع العناصر الأصغر من المحور على يساره، والعناصر الأكبر منه على يمينه.

**** التكرار (Recursion): **** تطبيق نفس الخطوات على الجانبين الأيمن والأيسر بشكل متكرر.

Quick Sort (3/5)

Here's the code for quicksort:

```
def quicksort(array):
    if len(array) < 2:
        return array  # Base case: arrays with 0 or 1 element are already "sorted."
    else:
        pivot = array[0]  # Recursive case
        less = [i for i in array[1:] if i <= pivot]  # Sub-array of all the elements less than the pivot
        greater = [i for i in array[1:] if i > pivot]  # Sub-array of all the elements greater than the pivot
        return quicksort(less) + [pivot] + quicksort(greater)

print quicksort([10, 5, 2, 3])
```

Page - 15



Quick Sort (4/5)

- **Best Case Recurrence:** $T(n) = 2T(\frac{n}{2}) + n$

Occurs when the pivot always divides the array into two equal halves

- **Worst Case Recurrence:** $T(n) = T(n - 1) + n$

Occurs when the **pivot is always the smallest or largest element**

- **Time Complexity:**

- ✓ Best Case: $\Omega(n \log n)$,
- ✓ Average Case: $\Theta(n \log n)$
- ✓ Worst Case: $O(n^2)$



**** خوارزمية ستراسن (Strassen's Algorithm): ****

*** تعتمد على تقسيم المصفوفة الكبيرة إلى مصفوفات أصغر.**

*** تستخدم معادلات جبرية لتقليل عدد عمليات الضرب المطلوبة من 8 إلى 7 فقط.**

**** الضرب التقليدي للمصفوفات: ****

*** التعقيد الزمني: $O(n^3)$**

The naive approach for matrix multiplication

- The naive approach directly implements the mathematical definition of matrix multiplication using three nested loops.

- Time Complexity: $O(n^3)$.

```
FUNCTION naiveMatrixMultiply(A, B, n)
    // A, B: input matrices of size n x n
    // n: dimension of matrices

    LET C be a new n x n matrix initialized to zeros

    FOR i FROM 0 TO n-1 DO
        FOR j FROM 0 TO n-1 DO
            FOR k FROM 0 TO n-1 DO
                C[i][j] = C[i][j] + A[i][k] * B[k][j]
            END FOR
        END FOR
    END FOR

    RETURN C
END FUNCTION
```

Page - 21

****من حيث الطريقة**:**

*الضرب التقليدي (Naive) يعتمد على ثلاث حلقات تكرار (Nested Loops) لتنفيذ التعريف الرياضي المباشر لضرب المصفوفات.
*خوارزمية ستراسن تعتمد على مبدأ "فرق تسد" (Divide and Conquer) وتقسيم المصفوفات.

****من حيث عدد العمليات التكرارية**:**

*الطريقة التقليدية تتطلب إجراء 8 عمليات ضرب للمصفوفات الفرعية.
*خوارزمية ستراسن تستخدم طريقة جبرية ذكية لتقليل العمليات إلى 7 عمليات ضرب فقط.

****من حيث الكفاءة والسرعة**:**

*الطريقة التقليدية أبطأ وتستغرق وقتاً أطول تعقيده $O(n^3)$.
*خوارزمية ستراسن أسرع في التعامل مع المصفوفات الضخمة حيث يقل التعقيد الزمني فيها إلى $O(n^{2.81})$.

Lecture 7

1. مشكلة تسلسل الوظائف (Job Sequencing Problem)

تُعد هذه المشكلة من أهم تطبيقات الخوارزميات الجشعة (Greedy Algorithms) لتعظيم الربح.

- الخطوات الأساسية (Steps):

- ترتيب الوظائف تنازلياً حسب الربح (Profit).
- تحديد أقصى موعد نهائي (Max Deadline) لإنشاء فترات زمنية (Time Slots).
- لكل وظيفة، ابحث عن أحدث فترة زمنية متاحة قبل موعدها النهائي المباشر وقم بتعيينها هناك.

- مثال (Example 1): لدينا 5 وظائف (J1 إلى J5) بأرباح ومواعيد نهائية مختلفة:

- يتم اختيار J2 (الربح 100) في الفتحة [1-0]، ثم J1 (الربح 60) في الفتحة [2-1]، ثم J3 (الربح 20) في الفتحة [3-2].
- الربح النهائي: 180.

- التعقيد الزمني (Time Complexity):

- تستغرق الخوارزمية $O(n^2)$ بسبب الحلقات المتداخلة (Outer loop للوظائف و Inner loop للبحث عن الفتحات الزمنية).

The classroom scheduling problem using Brute Force algorithm

- Classroom Scheduling: "Which Classes to Choose?"**

Decision: Include or exclude each class from the schedule.

Order doesn't matter - we only care about the **set** of classes.

Binary choice per class: Take it or leave it.

- Brute force complexity: $O(2^n)$

```
FUNCTION simple_brute_force(classes):
    best = [] // Will store our best schedule

    // Try including/excluding each class in all possible ways
    FOR include_A in [True, False]:
        FOR include_B in [True, False]:
            FOR include_C in [True, False]:
                // ... repeat for all classes

                current = []
                IF include_A: ADD class_A to current
                IF include_B: ADD class_B to current
                IF include_C: ADD class_C to current
                // ... for all classes

                IF no_overlaps(current):
                    IF length(current) > length(best):
                        best = current

    RETURN best
```

The classroom scheduling problem using a greedy algorithm

- The greedy algorithm for classroom scheduling has a time complexity of $O(n \log n)$ because of its two main steps. First, it must sort all the classes by their end time. Sorting n items using an efficient algorithm like Merge Sort or Quick Sort always takes $O(n \log n)$ time; this is a well-known computational cost for organizing any list of items through comparisons. Second, the algorithm makes a single pass through this now-sorted list to pick non-overlapping classes, which only takes $O(n)$ time.
- When we combine these steps, the total time is $O(n \log n) + O(n)$. For a large number of classes, the $O(n \log n)$ sorting step dominates and determines the overall complexity, making it much more efficient than a brute-force approach.

```
FUNCTION schedule_classes(classes):
    IF classes is empty:
        RETURN empty list

    // Step 1: Sort classes by ending time
    SORT classes BY end_time ASCENDING

    // Step 2: Initialize with first class
    selected_classes = [first_class]
    last_end_time = first_class.end_time

    // Step 3: Greedily select non-overlapping classes
    FOR each class in remaining_classes:
        IF class.start_time >= last_end_time:
            ADD class to selected_classes
            last_end_time = class.end_time

    RETURN selected_classes
```

Page - 15

2. الحد الأدنى لشجرة الامتداد (Minimum Spanning Tree - MST)

- تُذكر المحاضرة أن "الحد الأدنى لشجرة الامتداد" هو أحد التطبيقات الرئيسية للخوارزميات الجشعة.
- تعتمد الخوارزميات الجشعة في هذا النوع (مثل Kruskal أو Prim) على اختيار الحواف ذات الوزن الأقل خطوة بخطوة مع التأكد من عدم تكوين دورات (Cycles).

3. حقيقة الظهر (Knapsack Problem: Fractional vs 0/1)

توضح المحاضرة الفرق الجوهرى بين النوعين ومدى نجاح الخوارزمية الجشعة مع كل منهما:

- حقيقة الظهر الجزئية (Fractional Knapsack):
 - تنتج الخوارزمية الجشعة في الوصول للحل الأمثل.
 - الطريقة: يتم حساب نسبة (الربح ÷ الوزن) لكل عنصر، ثم ترتيب العناصر تنازلياً حسب هذه النسبة، وأخذ أكبر قدر ممكن من العناصر الأفضل.
 - التعقيد الزمني: $O(n \log n)$ بسبب خطوة الترتيب.

Fractional knapsack problem using brute force algorithm

- **Brute Force:** Guaranteed optimal but exponentially slow
- **Time Complexity:** $O(2^n)$

```
FUNCTION brute_force_knapsack(weights, values, capacity):
    n = length(weights)
    max_value = 0

    # Generate all possible subsets (2^n possibilities)
    FOR bitmask FROM 0 TO (2^n - 1):
        total_weight = 0
        total_value = 0

        # Check which items are included in this subset
        FOR i FROM 0 TO n-1:
            IF bitmask has i-th bit set:
                total_weight += weights[i]
                total_value += values[i]

        # If valid and better, update best solution
        IF total_weight <= capacity AND total_value > max_value:
            max_value = total_value

    RETURN max_value
```

Page - 19

● حقيبة الظهر 1/0 (Knapsack 1/0):

- تفشل الخوارزمية الجشعة في إيجاد الحل الأمثل.
- السبب أن اختيار العنصر الأعلى حالياً قد يمنعنا من اختيار عدة عناصر أقل سعراً لكن مجموعها الكلي أكبر.
- يُعد هذا النوع مثالاً على أن "الأفضل محلياً لا يضمن الأفضل عالمياً" في بعض المسائل.

4. مثال الجدول في شريحة 25 (Slide 25 Table)

Fractional knapsack problem using a greedy algorithm- Example (1)

● Time Complexity

Step	Operation	Time Complexity	Why
1	Calculate ratios	$O(n)$	One operation per item
2	Sort items	$O(n \log n)$	Comparison-based sorting
3	Greedy selection	$O(n)$	Single pass-through items
4	Final calculation	$O(1)$	Simple arithmetic

- **Total time:** $O(n \log n)$ - dominated by sorting

Page - 25



5. تحسين خوارزمية تسلسل الوظائف (Job Sequencing Improvement)

- النسخة الأساسية المعروضة في المحاضرة تعتمد على البحث المتكرر عن الفتحات الزمنية وتستغرق $O(n^2)$.
- التحسين المعروف (رغم عدم ذكره بالتفصيل في نص المحاضرة لكنه مطلوب ضمن سياق "التحسين") يعتمد على استخدام هيكل بيانات مجموعات الربط المنفصلة (Disjoint Set Union - DSU).
- هذا التحسين يسمح بإيجاد الفتحة الزمنية المتاحة في وقت شبه ثابت، مما يقلل التعقيد الإجمالي إلى $O(n \log n)$ (بسبب الترتيب) أو $O(n \alpha(n))$ للبحث عن الفتحات.

Job Sequencing with Deadline using greedy algorithm- Example (1)

- The job sequencing algorithm uses **nested loops**:
- **Outer Loop:**
 - ✓ Iterates through all n jobs (after sorting)
 - ✓ Processes each job one by one in profit-descending order
- **Inner Loop:**
 - ✓ For each job, finds the latest available time slot before its deadline
 - ✓ May need to check up to d time slots (where d is the maximum deadline)
- **Time Complexity: $O(n^2)$**

Page - 38



Lecture 8

1. مشكلة حقيبة الظهر 1/0 (Knapsack Problem 1/0) باستخدام البرمجة الديناميكية

* **طريقة الجدول (Tabulation):**

* تبدأ الخوارزمية بإنشاء جدول حجمه (عدد العناصر + 1) × (السعة القصوى + 1).

* يتم ملء الجدول صفًا بصف، حيث يمثل كل مربع أقصى قيمة يمكن تحقيقها باستخدام العناصر المتاحة حتى ذلك الصف وبسعة الحقيبة المتاحة لذلك العمود.

* النتيجة النهائية (أقصى ربح) توجد دائماً في الخانة الأخيرة من الجدول، أي في .

* **إيجاد العناصر المختارة:** لمعرفة العناصر الفعلية التي شكلت هذا الربح، نبدأ من الخانة الأخيرة ونعود للخلف في الجدول؛ إذا كانت القيمة تختلف عن الخانة التي فوقها، فهذا يعني أن العنصر الحالي تم اختياره.

0/1 Knapsack problem using dynamic programming

- How to find actual Knapsack Items:

i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

```

i=n, k=W
while i,k > 0
    if B[i,k] ≠ B[i-1,k] then
        mark the nth item as in the knapsack
        i = i-1, k = k-wi
    else
        i = i-1
    
```

The optimal knapsack should contain {1, 2}

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

Page - 40

التعقيد الزمني (Time Complexity):

*

الحل المباشر (Brute Force): يكون وهو غير فعال مع المدخلات الكبيرة. $O(2^n)$

البرمجة الديناميكية (DP): التعقيد الزمني هو ، حيث هو عدد العناصر و هي السعة القصوى للحقيبة.

0/1 Knapsack problem using dynamic programming

- The time complexity of the 0/1 Knapsack problem using dynamic programming is $O(NW)$, where:

N: represents the number of items available.

W: represents the maximum capacity of the knapsack.

* **طريقة الجدول والمنطق الرياضي:**

* يتم إنشاء مصفوفة (جدول) حيث و هما طول السلسلتين.

* إذا كان الحرفان متطابقين (): يتم إضافة 1 إلى قيمة الخانة القطرية السابقة، أي .

* إذا كان الحرفان غير متطابقين: يتم أخذ القيمة الأكبر من الخانة التي فوقها أو الخانة التي على يسارها، أي .

* **استخراج السلسلة:** بعد ملء الجدول، يتم استخراج الأحرف بالعودة من إلى البداية؛ عندما نجد تطابقاً (قيمة جاءت من القطر)، نقوم بتسجيل الحرف.

LCS using Brute Force

- What is the Longest Common Subsequence of X and Y?

X = A B C B

Y = B D C A B

- The brute-force method for LCS works as follows:

- Generate all possible subsequences of sequence X.
- Generate all possible subsequences of sequence Y.
- Compare every subsequence of X with every subsequence of Y.
- Find the longest subsequence that appears in both lists.
- **Time Complexity:** $O(2^M \times 2^N) = O(2^{M+N})$, where M and N are the lengths of the strings. For M=7, N=5, this is $2^{12} = 4096$ comparisons in the worst case.

LCS using DP

i	j	0	1	2	3	4	5
		Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0	0
1	A	0	0	0	0	1	1
2	B	0	1	1	1	1	2
3	C	0	1	1	2	2	2
4	B	0	1	1	2	2	3



LCS using DP

- LCS algorithm calculates the values of each entry of the array $c[m, n]$.
- So, what is the running time?
 - $O(m * n)$
 - Since each $c[i, j]$ is calculated in constant time, and there are $m * n$ elements in the array.

```

1. m ← length[X]
2. n ← length[Y]
3.
4. // Initialize tables
5. for i ← 0 to m do
6.   c[i, 0] ← 0
7. for j ← 0 to n do
8.   c[0, j] ← 0
9.
10. // Fill the DP tables
11. for i ← 1 to m do
12.   for j ← 1 to n do
13.     if xi = yj then
14.       c[i, j] ← c[i-1, j-1] + 1
15.       b[i, j] ← "↖" // Characters match, move diagonally
16.     else if c[i-1, j] ≥ c[i, j-1] then
17.       c[i, j] ← c[i-1, j]
18.       b[i, j] ← "↑" // Move upward (skip character from X)
19.     else
20.       c[i, j] ← c[i, j-1]
21.       b[i, j] ← "←" // Move leftward (skip character from Y)
22.
23. return c and b
    
```


Breadth-first-search (BFS)

- BFS appears to have $O(|V| \times |E|)$ complexity due to nested loops but actually achieves $O(|V| + |E|)$ through careful tracking.
- The outer while loop runs at most $|V|$ times since each vertex enters the queue exactly once, thanks to the visited set preventing repeats.
- The inner for loop examines edges, but crucially, each edge is examined only once from its source vertex or twice in undirected graphs.
- The total edge examinations sum to $O(|E|)$ across all vertices, not multiplied per vertex. **Thus, total work equals vertex operations ($|V|$) plus edge operations ($|E|$), making BFS efficient for large graphs.**

```
procedure BFS(G, s):
    initialize empty queue Q
    initialize visited set S ← ∅

    S.insert(s)           // Mark source visited
    Q.enqueue(s)          // Enqueue for expansion

    while Q ≠ ∅:
        v ← Q.dequeue()

        for each neighbor u ∈ Adj[v]:
            if u ∉ S:      // State space pruning
                S.insert(u) // Prevent future rediscovery
                Q.enqueue(u) // Schedule for expansion
```

Page - 35



Depth First Search (DFS)

- DFS time complexity is $O(|V| + |E|)$, identical to BFS. Each vertex is visited exactly once, contributing $O(|V|)$ operations to the total time.
- Each edge is examined once when its source vertex is processed, adding $O(|E|)$ operations.
- The visited set prevents re-processing vertices, ensuring linear growth relative to graph size.

```
DFS_Simple(start):
    stack = [start]
    visited = set()

    while stack:
        current = stack.pop()

        if current not visited:
            mark current as visited

            for each neighbor of current:
                if neighbor not visited:
                    stack.push(neighbor)
```

Page - 48

